

COLLEGE OF ENGINEERING, GUINDY

Chennai - 600025



DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY - IST

Assignment Report on

Implementation of RMI on AWS Cloud

in

**DISTRIBUTED SYSTEMS AND COMPUTING –
IT23601**

Submitted by

Mohan Vishnu Kumar – 2023115026

Implementation of Remote Method Invocation (RMI) in a Cloud Environment (AWS)

1. Introduction

Remote Method Invocation (RMI) is a Java-based distributed computing technology that allows an object running on one machine to invoke methods on an object running on another machine. RMI hides the complexities of network communication and makes remote method calls appear like local method calls.

In this experiment, a Java RMI-based distributed application was implemented and deployed on a cloud environment (AWS EC2). The client remotely accesses the methods of a server-side object and receives the computed results.

2. Objective of the Experiment

The objectives of this experiment are:

1. To understand the working of Java RMI.
2. To implement a distributed application using RMI.
3. To host the RMI server on a cloud platform.
4. To enable remote method invocation by the client.
5. To handle runtime and network errors.
6. To observe real-time distributed communication.

3. Tools and Technologies Used

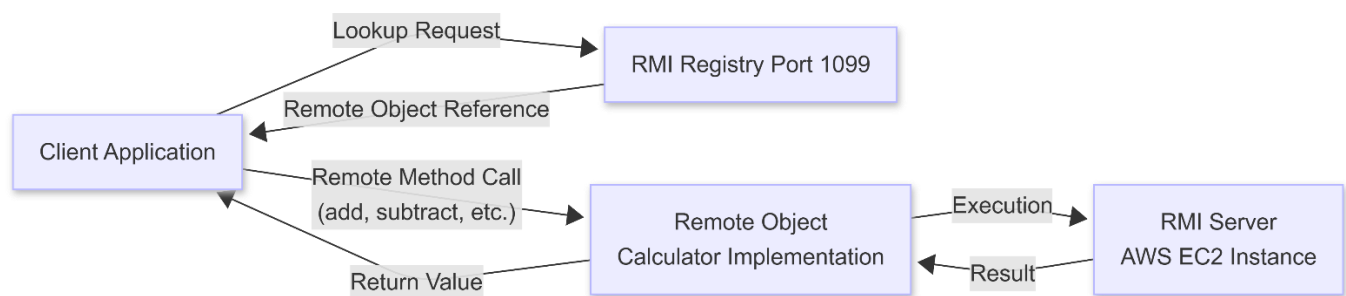
Component	Description
Programming Language	Java 23
Technology	Java RMI
Cloud Platform	AWS EC2
Operating System (Server)	Ubuntu 22.04

Operating System (Client)	Windows
Protocol	TCP/IP
IDE / Editor	VS Code, Nano
JDK	OpenJDK

4. System Architecture

The RMI system consists of the following components:

- 1. Remote Interface** - Defines the methods that can be invoked remotely.
- 2. Remote Implementation** - Implements the remote interface and provides the actual logic.
- 3. RMI Registry** - Acts as a naming service that allows clients to locate remote objects.
- 4. Client** - Requests the registry for the remote object and invokes methods.



5. Working of RMI in This Project

1. The RMI server is started on an AWS EC2 instance.
2. The server creates a remote object.
3. The remote object is registered in the RMI registry.
4. The client connects to the registry using the public IP.
5. The client looks up the remote object.
6. The client invokes remote methods.
7. The server executes the method.
8. The result is sent back to the client.

6. Implementation Details

6.1 Remote Interface

The remote interface declares the methods that can be invoked remotely. It extends the Remote interface, and all methods throw RemoteException.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

// Remote interface
public interface Calculator extends Remote {

    int add(int a, int b) throws RemoteException;

    int subtract(int a, int b) throws RemoteException;

    int multiply(int a, int b) throws RemoteException;

    double divide(int a, int b) throws RemoteException;
}
```

6.2 Remote Implementation

This class implements the remote interface and extends UnicastRemoteObject. It contains the actual logic for the calculator operations.

Server-side logs are generated for every request to track execution.

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

// Implementation of remote interface
public class CalculatorImplementation extends UnicastRemoteObject implements
Calculator {

    // Constructor
    protected CalculatorImplementation() throws RemoteException {
        super();
    }

    public int add(int a, int b) throws RemoteException {
        System.out.println("Adding " + a + " with " + b);
        return a + b;
    }
}
```

```

    public int subtract(int a, int b) throws RemoteException {
        System.out.println("Subtracting " + b + " from " + a);
        return a - b;
    }

    public int multiply(int a, int b) throws RemoteException {
        System.out.println("Multiplying " + a + " with " + b);
        return a * b;
    }

    public double divide(int a, int b) throws RemoteException {
        if (b == 0) {
            System.out.println("Division by zero attempted with " + a + " and
" + b);
            throw new RemoteException("Division by zero not allowed");
        }
        System.out.println("Dividing " + a + " by " + b);
        return (double) a / b;
    }
}

```

6.3 RMI Server

The server:

- Creates the remote object
- Starts the RMI registry
- Binds the object to the registry
- Waits for client requests

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class RMIServer {
    public static void main(String[] args) {
        try {
            System.setProperty("java.rmi.server.hostname", "13.48.249.5");

            CalculatorImplementation calc = new CalculatorImplementation();

            // Create registry on port 1099
            Registry registry = LocateRegistry.createRegistry(1099);

            // Bind the object
            registry.rebind("CalculatorService", calc);
        }
    }
}

```

```

        System.out.println("RMI Server is running...");
    } catch (Exception e) {
        System.out.println("Server error: " + e);
        e.printStackTrace();
    }
}
}

```

6.4 RMI Client

The client:

- Connects to the RMI registry
- Looks up the remote object
- Calls the remote methods
- Displays the returned results

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class RMIClient {
    public static void main(String[] args) {
        try {
            // Connect to RMI registry
            Registry registry = LocateRegistry.getRegistry("13.48.249.5",
1099);

            // Lookup the remote object
            Calculator calc = (Calculator)
registry.lookup("CalculatorService");

            // Call remote methods
            System.out.println("Addition: " + calc.add(5, 3));
            System.out.println("Subtraction: " + calc.subtract(10, 4));
            System.out.println("Multiplication: " + calc.multiply(6, 7));
            System.out.println("Division: " + calc.divide(20, 5));

        } catch (Exception e) {
            System.out.println("Client error: " + e.getMessage());
        }
    }
}

```

7. Cloud Deployment (AWS EC2)

To make the application truly distributed, the RMI server was deployed on AWS EC2.

Steps Followed:

1. Created a free-tier EC2 instance.
2. Installed Java on the server.
3. Uploaded RMI source files.
4. Opened port 1099 and dynamic ports.
5. Compiled Java files.
6. Started the RMI server.
7. Connected the client using the public IP.

Screenshots of the work:

AWS EC2 instance:

The screenshot displays the AWS Management Console interface for EC2 instances. At the top, the navigation bar shows the AWS logo, a search bar, and the user's profile (Mohan). The main header indicates the current page is 'EC2 > Instances'. Below this, a summary bar shows 'Instances (1/1)' with a search bar and filters. A table lists the instance details:

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public
RMI-Server	i-07a03c48ab1990735	Running	t3.micro	3/3 checks passed	View alarms +	eu-north-1b	ec2-13

Below the table, the details for the selected instance 'i-07a03c48ab1990735 (RPC-Server)' are shown. The 'Details' tab is active, displaying the following information:

- Instance ID:** i-07a03c48ab1990735
- Public IPv4 address:** 13.48.249.5 | [open address](#)
- Private IPv4 addresses:** 172.31.42.178
- Instance state:** Running
- Public DNS:** ec2-13-48-249-5.eu-north-1.compute.amazonaws.com | [open address](#)

Inbound-rules:

The screenshot shows the AWS Management Console interface for a security group. The top navigation bar includes the AWS logo, a search bar, and the user's name 'Mohan'. The breadcrumb trail indicates the path: EC2 > Security Groups > sg-0eb67105171d3e8bf - launch-wizard-1. The main heading is 'sg-0eb67105171d3e8bf - launch-wizard-1' with an 'Actions' button. Below this is a 'Details' section with a grid of information: Security group name (launch-wizard-1), Security group ID (sg-0eb67105171d3e8bf), Description (launch-wizard-1 created 2026-01-15T 15:42:18.052Z), VPC ID (vpc-033ff817dccb41b1b), Owner (010265788133), Inbound rules count (5 Permission entries), and Outbound rules count (1 Permission entry). Below the details are tabs for 'Inbound rules', 'Outbound rules', 'Sharing', 'VPC associations', and 'Tags'. The 'Inbound rules' tab is active, showing a table of 5 rules. The table has columns for rule ID, IP version, type, protocol, port range, source, and description. The rules are: Custom TCP (ports 1024-65535), HTTPS (port 443), SSH (port 22), Custom TCP (port 1099), and HTTP (port 80), all with source 0.0.0.0/0.

Details

Security group name launch-wizard-1	Security group ID sg-0eb67105171d3e8bf	Description launch-wizard-1 created 2026-01-15T 15:42:18.052Z	VPC ID vpc-033ff817dccb41b1b
Owner 010265788133	Inbound rules count 5 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules (5)

Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
af5bd518d4bf13e29	IPv4	Custom TCP	TCP	1024 - 65535	0.0.0.0/0	-
ae06be5253f2a2a15	IPv4	HTTPS	TCP	443	0.0.0.0/0	-
4a6e2520987e05b8	IPv4	SSH	TCP	22	0.0.0.0/0	-
7d76dea64b2ca152	IPv4	Custom TCP	TCP	1099	0.0.0.0/0	-
bd0505caaf36c8a65	IPv4	HTTP	TCP	80	0.0.0.0/0	-

Starting the server:

```
PS C:\Users\curvy\Downloads> ssh -i rpc-key.pem ubuntu@13.48.249.5
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Jan 15 18:27:46 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   38.9% of 6.71GB Processes:     113
Memory usage: 27%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.42.178

Expanded Security Maintenance for Applications is not enabled.

88 updates can be applied immediately.
48 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Thu Jan 15 18:19:30 2026 from 106.192.161.61
ubuntu@ip-172-31-42-178:~$ java RMIServer
RMI Server is running...
|
```

8. Error Handling

Error handling was implemented on both client and server.

Server-Side:

- Prevents division by zero
- Logs all client requests
- Throws meaningful exceptions

Client-Side:

- Handles registry lookup failures
- Handles connection issues
- Displays friendly error messages

9. Output

The client successfully invoked remote methods from the AWS-hosted server.

Client-side output:

```
PS C:\Users\curvy\Desktop\RPC&RMI-DS_Assignment1\RMI> java RMIClient
Addition: 8
Subtraction: 6
Multiplication: 42
Division: 4.0
```

Server-side logs:

```
ubuntu@ip-172-31-42-178:~$ java RMIServer
RMI Server is running...
Adding 5 with 3
Subtracting 4 from 10
Multiplying 6 with 7
Dividing 20 by 5
|
```
